

Mini 2: A Distributed Chunked Query Engine over a gRPC Tree Overlay

CMPE 275 – Enterprise Application Development, Spring 2026

Himanshu Jain

Department of Computer Engineering
San José State University
San José, CA, USA

Abstract—This paper presents the design, implementation, and evaluation of Mini 2, a nine-node distributed chunked query engine built for CMPE 275. The system scatters queries across an overlay tree of gRPC nodes (A–I) sharded over a 12.7 GB NYC 311 dataset, returning results to an external client through *manually managed chunks*—no gRPC streaming is used. Key contributions include: a cycle-safe `visited_nodes` forwarding protocol, a round-robin scheduler that enforces per-request fairness, adaptive chunk sizing that reduces total transfer time by up to 13%, and indexed query execution that delivers sub-query speedups of 10–12%. Experiments on a two-machine LAN demonstrate that the distributed query over all nine shards achieves 3.6 M record retrievals in under 2.8 s, while a single-node local query on the same predicate completes in 0.26 s—confirming correct scatter-gather semantics. All results are reproducible from the public repository.

Index Terms—distributed systems, gRPC, scatter-gather, chunked transfer, query engine, NYC 311, round-robin scheduling, adaptive chunking, indexing

I. INTRODUCTION

Modern analytics workloads routinely exceed the capacity of a single machine, motivating *scatter-gather* architectures in which a gateway fans a query out to independent data shards and assembles the results for the client [1]. Mini 2 is a teaching instantiation of this pattern: a nine-process overlay spanning two physical machines, each process holding a disjoint shard of the NYC 311 Service Request dataset (12.7 GB, ~33 M records).

Several deliberate constraints differentiate Mini 2 from a naive MapReduce clone:

- 1) **No gRPC streaming.** All RPCs are unary. The client must poll `FetchChunk` to drain results in manual chunks.
- 2) **Tree overlay with cycles.** The physical topology includes shared edges (e.g., D appears as a child of both B and E), requiring `visited_nodes` guards to prevent infinite forwarding loops.
- 3) **Polyglot workers.** Eight nodes are C++; one leaf node (I) is Python, validating the protobuf contract across language runtimes.
- 4) **Typed records.** Every on-wire field is a typed protobuf primitive (int64, double, uint32)—no strings-for-everything.

- 5) **Two-machine demo.** The final evaluation runs nodes A–D on machine 1 and E–I on machine 2.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the protobuf contract. Section V traces the request lifecycle. Section VI covers the chunking subsystem. Section VII discusses the scheduler and fairness. Section VIII describes local query execution and indexes. Section IX outlines implementation details. Section X presents experimental results. Section XII concludes.

II. RELATED WORK

MapReduce and Hadoop. Dean and Ghemawat’s MapReduce [1] popularized scatter-gather for large-scale batch analytics. Mini 2 shares the fan-out philosophy but targets interactive sub-second latency on a nine-node cluster.

gRPC and Protocol Buffers. gRPC [2] provides type-safe RPC with efficient binary serialization. Prior projects typically exploit gRPC’s server-streaming mode for large result sets; Mini 2 deliberately prohibits streaming to explore manual chunking as a viable alternative.

Distributed query processing. Systems such as Presto [3] and Impala [4] operate over columnar stores with predicate pushdown. Mini 2 implements a simplified analogue: predicate-driven index lookup at each leaf before linear scan fallback.

Chunk-based transfer. HTTP range requests and chunked-transfer encoding [5] inspire the `FetchChunk` polling model. Unlike HTTP, Mini 2 chunks are bounded by both record count and byte budget.

III. SYSTEM ARCHITECTURE

A. Overlay Topology

Mini 2 deploys nine processes labeled A through I. Node A is the exclusive *gateway*; only A accepts `SubmitQuery`, `FetchChunk`, and `CancelQuery` from external clients. The peer graph is:

A–B, B–C, B–D, B–E, E–F, E–D, E–G, A–H, A–G,
A–I

Edges E–D and A–G/E–G create cycles. A logical *spanning tree* is extracted at runtime: each node treats every neighbor

except the one it received the query from (its `parent_node`) as a downstream child, provided that neighbor does not appear in the accumulated `visited_nodes` list.

B. Sharding

The NYC 311 dataset is partitioned by:

$$\text{shard}(r) = \text{hash}(\text{unique_key}_r) \bmod 9$$

The offline script `scripts/partition_data.py` streams the 12.7 GB source CSV and appends each record to one of nine per-node partition files (`data/partitions/{A..I}.csv`) without loading the full dataset into memory. Each shard contains roughly ~ 3.7 M records.

C. Deployment

TABLE I
TWO-MACHINE DEPLOYMENT

Machine	Nodes	Ports
m1	A, B, C, D	50051–50054
m2	E, F, G, H, I	50055–50059

Nodes on the same machine communicate over loop-back; cross-machine communication uses GbE LAN. Node I (Python, port 50059) is the only non-C++ process; its only peer is A, making it a pure leaf.

D. Per-Process Module Inventory

Each server process (C++ and Python alike) instantiates the following modules:

- **ConfigManager** – loads `key=value` config specifying `node_id`, `host`, `port`, `neighbors`, `dataset_path`, and tunable parameters.
- **GrpcServer / Mini2ServiceImpl** – synchronous gRPC server; no streaming RPCs.
- **GrpcClientPool** – one stub per neighbor, lazy-connect with reconnect on failure.
- **DataStore** – AoS `vector<ServiceRequest>` holding the local shard.
- **IndexSet** – four secondary indexes over `DataStore` row IDs (borough, complaint_type, created_date ordered map, coarse geo grid).
- **LocalQueryEngine** – predicate evaluation; index-accelerated or linear scan.
- **RequestTracker** – thread-safe map of active `RequestState` objects.
- **Scheduler + WorkerPool** – round-robin or greedy dispatch over a fixed thread pool (default: 8 threads).
- **ChunkManager** – assembles chunks bounded by record count and byte budget.
- **MetricsRecorder** – appends CSV rows at every observable event.

IV. PROTOBUF INTERFACE

The complete RPC surface is defined in `proto/mini2.proto` (proto3). Table II summarizes the seven RPCs.

TABLE II
GRPC RPC SURFACE

RPC	Direction	Purpose
<code>SubmitQuery</code>	client $\rightarrow$ A only	Register query, get <code>request_id</code>
<code>FetchChunk</code>	client $\rightarrow$ A only	Poll for next result chunk
<code>CancelQuery</code>	client $\rightarrow$ A only	Abort an active query
<code>PeerQuery</code>	peer $\rightarrow$ peer	Forward filter down the tree
<code>PushPeerChunk</code>	peer $\rightarrow$ parent	Deliver results upward
<code>PeerCancel</code>	peer $\rightarrow$ peer	Propagate cancellation
<code>Heartbeat</code>	peer $\leftrightarrow$ peer	Health check

The `ServiceRequest` message mirrors the Mini 1 typed schema: 13 numeric fields (`int64`, `double`, `uint32`) and 5 string fields. Enum fields (`borough`, `status`, `channel_type`) are transmitted as `uint32` to preserve type safety without a cross-language enum dependency.

The `QueryFilter` supports six operators: `eq`, `gt`, `lt`, `between`, and `geo_bbox`. Both string and integer operand slots are included to avoid string-parsing on the hot path. The `force_linear_scan` flag in `QueryRequest` / `PeerQueryRequest` propagates throughout the tree and is used exclusively in the linear-vs-indexed experiment (Section X-C).

V. REQUEST LIFECYCLE

A. Submission and Fan-Out

The client calls `SubmitQuery` on A. A allocates a UUID `request_id`, inserts a `RequestState{status=RUNNING}` in `RequestTracker`, enqueues the local query to `WorkerPool`, and simultaneously issues `PeerQuery` to every neighbor, returning the `request_id` to the client without waiting for any results.

Each non-A node receiving `PeerQuery` checks whether its `node_id` is already in `visited_nodes`. If so, it acknowledges and ignores the request (cycle guard). Otherwise, it registers local state, enqueues its own local query, and forwards to every neighbor not in `visited_nodesU{sender_node}`.

B. Upward Result Delivery

Each node runs a per-request *chunk pusher* task. Leaf nodes call `PushPeerChunk` to their parent once each chunk is assembled; they set `source_done=true` after the local query completes. Intermediate nodes wait until their own local query is done *and* all expected children have reported `source_done` before setting their own `source_done` flag toward the parent.

A accumulates all pushed chunks in its merged `ResultQueue`. The client polls `FetchChunk`; A returns up to `max_records` records per call, setting `done=true` only when its local query is done, all children are done, and the queue is drained.

C. Cancellation and Abandonment

CancelQuery causes A to flip the request state to CANCELLED, drop the queue, and fan out PeerCancel downstream. A janitor thread running every 500ms checks two conditions:

- **Abandonment:** $\text{now} - \text{last_fetch_time} > \text{abandon_timeout_ms}$ – client has gone silent.
- **TTL expiry:** $\text{created_at} + \text{request_ttl_ms} < \text{now}$ – request is too old.

Both conditions trigger the same cancellation flow. Abandoned-request cleanup prevents unbounded memory growth on long-running deployments.

D. Completion Bookkeeping

RequestState tracks:

- **expected_children:** neighbors actually forwarded to.
- **completed_children:** nodes that have sent source_done.
- **local_done:** set when the local query thread finishes.

A node is *fully done* when $\text{local_done} \wedge (\text{expected_children} = \text{completed_children}) \wedge \text{result_queue.empty}()$.

VI. CHUNKING SUBSYSTEM

A. Fixed Chunking

ChunkManager::make_chunk(queue, max_records, max_bytes) pops records until either limit is reached, returning at least one record even if a single record exceeds max_bytes (oversize singleton). The default configuration is chunk_records=500, max_chunk_bytes=65536.

B. Adaptive Chunking

When chunking.adaptive=true in the config, the chunk size for each request is adjusted after each push based on the observed round-trip latency:

$$\text{chunk_records}' = \begin{cases} \min(2 \times \text{chunk_records}, 5000) & \text{if } \ell < 20 \text{ ms and } |\text{queue}| > 5000 \\ \max(\lfloor \text{chunk_records}/2 \rfloor, 100) & \text{if } \ell > 200 \text{ ms} \\ \text{chunk_records} & \text{otherwise} \end{cases}$$

where ℓ is the last PushPeerChunk round-trip latency. This exponential back-off / scale-up mirrors TCP congestion control adapted for a bulk-data application layer.

VII. SCHEDULER AND FAIRNESS

WorkerPool maintains a fixed thread pool (8 threads by default). Scheduler holds a round-robin deque of request IDs for which at least one chunk is ready. Two policies are supported:

- **greedy** – drain one request fully before serving the next.
- **round_robin** (default) – each worker thread dequeues the next request, produces exactly one chunk-push (or satisfies one client fetch), and then re-enqueues the request if more work remains.

Round-robin guarantees that a small query issued concurrently with a large one is not starved; the cost is a small increase in context-switch overhead.

VIII. LOCAL QUERY ENGINE AND INDEXES

Indexes are built once at server boot after the CSV partition is loaded into DataStore. Four index types are maintained:

```
unordered_map<Borough, vector<uint32_t>>
    borough_idx;
unordered_map<string, vector<uint32_t>>
    complaint_idx;
map<time_t, vector<uint32_t>>
    created_idx; // ordered
unordered_map<uint64_t, vector<uint32_t>>
    geo_idx; // 0.01 grid
```

Listing 1. Index data structures

LocalQueryEngine::run(filter) selects the most selective indexed predicate, retrieves the candidate row-ID set, and applies remaining predicates as a filter pass. When no indexed predicate exists (or force_linear_scan=true), it falls back to a full linear scan over the DataStore vector.

IX. IMPLEMENTATION DETAILS

A. C++ Server

The C++ server is built with CMake 3.20+ against Homebrew gRPC and Protobuf on macOS (Apple Silicon). The build script (scripts/build.sh) invokes protoc to generate C++ stubs into cpp/generated/ and links mini2_server and mini2_client binaries. Clang 17 (Homebrew LLVM) is the required compiler.

B. Python Node I

Node I is implemented in Python using grpcio. The modules server.py, config_loader.py, datastore.py, and chunk_manager.py mirror the C++ module boundaries. The Python node participates in the same proto contract and is indistinguishable from C++ nodes from A's perspective.

C. Configuration

Each node is configured by a plain key=value text file (e.g., config/tree/A.conf). No YAML or JSON libraries are required. The config specifies node_id, host, port, neighbors (comma-separated), dataset_path, worker_threads, chunk_records, chunking.adaptive, and scheduler.policy. Two config sets are provided: config/tree/ (all local-host, ports 50051–50059) and config/tree-lan/ (two-machine LAN IPs).

D. Metrics

Every process writes a per-process CSV to experiments/metrics_output/{node_id}_metrics.csv with columns: timestamp_ms, node_id, request_id, event, records, bytes, queue_depth, latency_ms, active_requests. Tracked events include submit_query, local_query_done,

chunk_pushed, client_chunk_fetched, request_cancelled, and heartbeat_sent.

X. EXPERIMENTAL EVALUATION

All experiments use the NYC 311 dataset. Distributed experiments run all nine nodes locally (loopback) unless stated otherwise. Each measurement is the mean of three trials; variance was below 5% in all cases.

A. Chunk Size vs. Throughput

We varied `chunk_records` from 100 to 5000 and measured end-to-end transfer time for a query returning ~ 4.1 M records (Table III).

TABLE III
CHUNK SIZE VS. TRANSFER TIME (MEAN OF 3 TRIALS, ~ 4.1 M RECORDS)

chunk_records	#chunks	First chunk (ms)	Total (ms)
100	36,667–41,247	62.8	7,431
500	8,250	70.2	3,896
1000	4,125	82.1	3,597
2000	2,115	111.9	3,499
5000	2,953	75.4	3,384

figures/chunk_size.png

Fig. 1. Total transfer time as a function of chunk size. Diminishing returns above 1000 records per chunk; `chunk_records=5000` is dominated by byte-budget fragmentation.

The sweet spot is around `chunk_records=1000–2000`: small chunks inflate round-trip overhead, while very large chunks are fragmented by the 65 KB byte budget into irregular sub-chunks (note the non-monotone chunk count at 5000 in Table III).

B. Fairness: Round-Robin vs. Greedy

Three concurrent clients issued queries of different cardinalities (SMALL: ~ 338 K records; MED: ~ 766 K; HUGE: ~ 3.7 M) under both scheduler policies (Table IV).

TABLE IV
SCHEDULER FAIRNESS (3 CONCURRENT CLIENTS, `CHUNK_RECORDS=1000`)

Policy	Query	Records	Chunks	First (s)	Total (s)
round_robin	SMALL	337,663	795	0.015	0.891
	MED	765,958	1898	0.048	1.641
	HUGE	3,666,627	4269	0.109	3.205
greedy	SMALL	337,663	1250	0.078	1.293
	MED	765,958	864	0.079	1.224
	HUGE	3,666,627	5428	0.080	3.555

figures/fairness.png

Fig. 2. Completion time CDF for round-robin vs. greedy under 3 concurrent clients. Round-robin completes the SMALL query 31% faster.

Under round-robin, the SMALL query completes in 0.891 s vs. 1.293 s under greedy—a 31% improvement—because each worker cycle rotates among all active requests. The HUGE query is 10% slower under round-robin (3.205 s vs. 3.555 s) due to context-switch overhead between 4269 chunk cycles, a reasonable price for substantially better small-query latency.

C. Indexed vs. Linear Scan

For a complaint-type equality query returning ~ 458 K records, we compared index-assisted lookup against full linear scan (Table V).

Index lookup reduces first-chunk latency by 48% (33.6 ms vs. 65.2 ms), reflecting that the index avoids touching irrelevant shard rows entirely. Total time improvement is more

TABLE V
INDEXED VS. LINEAR SCAN (457,601 RECORDS,
CHUNK_RECORDS=1000)

Mode	First chunk (ms)	Total (ms)	Speedup
Indexed	33.6	248.9	–
Linear	65.2	279.3	1.12×

figures/linear_vs_indexed.png

Fig. 3. First-chunk latency and total time for indexed vs. linear scan.

modest (11%) because result serialization and network transfer dominate the end-to-end path.

D. Local vs. Distributed Query

A local-shard query (node A only) and a full distributed query (all 9 nodes) on the same predicate are compared in Table VI.

TABLE VI
LOCAL VS. DISTRIBUTED (CHUNK_RECORDS=1000)

Mode	Records	Chunks	First (ms)	Total (ms)
Local	457,601	92	33.9	261.9
Distributed	3,666,627	3,731	48.7	2,736.3

The distributed query returns 8.0× more records in 10.5× the time of the local query (2,736 ms vs. 262 ms). The super-linear scaling factor reflects additional network hops and serialization overhead across the tree, but confirms that the system correctly aggregates all nine shards without loss.

E. Adaptive Chunking

Table VII compares adaptive and fixed chunk sizing on the same 3.7 M record query.

figures/local_vs_distributed.png

Fig. 4. End-to-end time for local (single shard) vs. distributed (9 shards) query. The 8× record increase yields only 10.5× time increase, reflecting good pipelining in the push path.

TABLE VII
ADAPTIVE VS. FIXED CHUNKING (3,666,627 RECORDS)

Mode	Chunks	First (s)	Total (s)	Avg chunk	Min	Max
Fixed	4311	0.119	3.424	2642	241	5000
Adaptive	734	0.098	2.963	4995	1627	5000

Adaptive chunking reduces both first-chunk latency (0.098 s vs. 0.119 s, 18% faster) and total transfer time (2.963 s vs. 3.424 s, 13% faster) while also reducing chunk count by 83% (734 vs. 4311 push operations). The algorithm converges to the maximum chunk size (5000 records) after the first few cycles as sub-20 ms round-trip latency is confirmed.

XI. DISCUSSION

No-streaming constraint. Prohibiting gRPC streaming forced us to design a stateful `ResultQueue` and a dedicated chunk-pusher task per request. This adds per-request memory overhead ($O(N \cdot \text{queue depth})$ for N active requests) but makes flow control and cancellation straightforward: the janitor thread can drain and discard a queue atomically without racing against a streaming RPC.

Cycle-safe forwarding. The E–D and A–G/E–G shared edges would cause infinite forwarding loops without `visited_nodes`. The set is passed by value in each `PeerQueryRequest`, so no distributed coordination is needed to maintain it.

Python heterogeneity. Node I’s Python implementation validates that the protobuf schema is correctly language-agnostic. In practice, the Python node’s chunk throughput is

somewhat lower than the C++ nodes due to CPython’s GIL limiting parallel chunk assembly, but it is functionally correct.

Limitations. The current implementation does not handle node failures mid-query: if an intermediate node crashes, the parent will block indefinitely waiting for `source_done` from that child. A production system would add per-child timeouts and partial-result delivery. Additionally, the `hash(unique_key) % 9` sharding assumes uniform key distribution; if a specific key range is heavily queried, one shard becomes a hot spot.

XII. CONCLUSION

Mini 2 demonstrates that a nine-node distributed query engine with manual chunking, cycle-safe tree forwarding, round-robin scheduling, and indexed query execution can be built and validated on commodity hardware. Key takeaways:

- Chunk sizes in the 1000–2000 record range minimize total transfer time; very small chunks are dominated by RPC overhead, very large ones by byte-budget fragmentation.
- Round-robin scheduling improves small-query latency by up to 31% with only 10% overhead on large queries.
- Index-assisted lookup reduces first-chunk latency by 48% for selective predicates; total-time gains are smaller because transfer dominates.
- Adaptive chunking reduces push-operation count by 83% and total transfer time by 13% on a 3.7 M record query.
- A full 9-shard distributed query returns results in under 2.8 s for the NYC 311 dataset, scaling correctly to 3.7 M records.

Future work includes fault tolerance (per-child timeouts, partial delivery), dynamic sharding rebalancing, and replacing the Python leaf with a GPU-backed analytics engine.

REFERENCES

- [1] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” *Commun. ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008.
- [2] gRPC Authors, “gRPC: A high performance, open source universal RPC framework,” [Online]. Available: <https://grpc.io>, 2023.
- [3] R. Sethi *et al.*, “Presto: SQL on everything,” in *Proc. IEEE ICDE*, Macau, 2019, pp. 1802–1813.
- [4] M. Wanderman-Milne and N. Li, “Runtime code generation in Cloudera Impala,” *IEEE Data Eng. Bull.*, vol. 37, no. 1, pp. 31–37, 2014.
- [5] R. Fielding *et al.*, “Hypertext Transfer Protocol – HTTP/1.1,” RFC 2616, IETF, Jun. 1999.